Inf2-SEPP 2025-26

Coursework 2

Creating a software design for an events app

SAMPLE SOLUTION

1 Task 1: Ambiguities

We list below the following ambiguities. This is not an exhaustive list, but it does help clarify certain decisions in the class and sequence diagrams.

- How does pre-registration of students/admin staff work? Assumption: We assume the system will be launched with the relevant students/admin staff, and hence we do not require a way to add them.
- What exactly is "University login information"?
  Assumption: It's a username + password. For EP's we assume the username is just their email (since those are by default unique), and they must also define their password on registration. For students and admin staff, the username and password would be whatever the university had assigned them when they were pre-registered (depends on their university system, which is external to ours).
- Shall EPs provide only their name or also the organisation name?
  Assumption: Both.
- How would the system check that an account for the EP does not already exist?
  Assumption: It would check: 1) that an account with the provided email address does not already exist; 2) that an account with the same organisation name and business number does not already exist (as an organisation should have a single account).

- What exactly are the possible event types?
  Assumption: Use only the ones mentioned in the requirements: music, theatre, dance, movie, sports. Others could be added relatively easily.
- Should performances and bookings be directly removed from the system when cancelled?
  Assumption: According to the provided requirements, in theory it would be fine to remove it completely. However, in practice it would be a smart idea to keep it, e.g. for reporting purposes in the future or to just keep a record of everything that was created in the system. We assume we will keep cancelled performances / bookings as objects, but will use a "status" attribute to keep track if they're cancelled or not.
- What should the statuses for performances and bookings be?
  Assumption: Performances will have the "ACTIVE" and "CANCELLED" statuses, and bookings will have the "ACTIVE", "CANCELLEDBYSTUDENT", "CANCELLEDBYPROVIDER" and "PAYMENTFAILED" statuses.
- Should the event status also be tracked?
  Assumption: No. Users search for performances and not events, which means if all performances of an event are cancelled then that event already won't appear as a result of the search. As such, a status for it isn't necessary.
- When an EP cancels a ticketed performance that has yet to have any bookings, do they still need to specify a cancellation message and request refunds from the payment system?
  Assumption: No, it doesn't make sense to specify a message or request refunds if there's no one to send them to.
- After a sponsorship, should the system keep both the original price and the discounted price?
  Assumption: Yes. (But for the coursework, the alternative will also be accepted.)
- It's not specified that a student needs to have an email or phone number, but they are required for payment. Should they be required for all students?
  Assumption: Yes
- How does payment work?
  Assumption: When appropriate, our system can send the Payment System a payment request, specifying the payment amount and some details about the transaction. This redirects the user to the payment system (which means away from our system). Then (again externally to our system) the money goes to the university accounts and admin staff need to then pay the EP the appropriate amount. Refunds work similarly.
- How does sponsorship work?
  Assumption: Admin staff can use our system to specify a performance ID and an amount to sponsor it by. All tickets booked from then on will be reduced in price

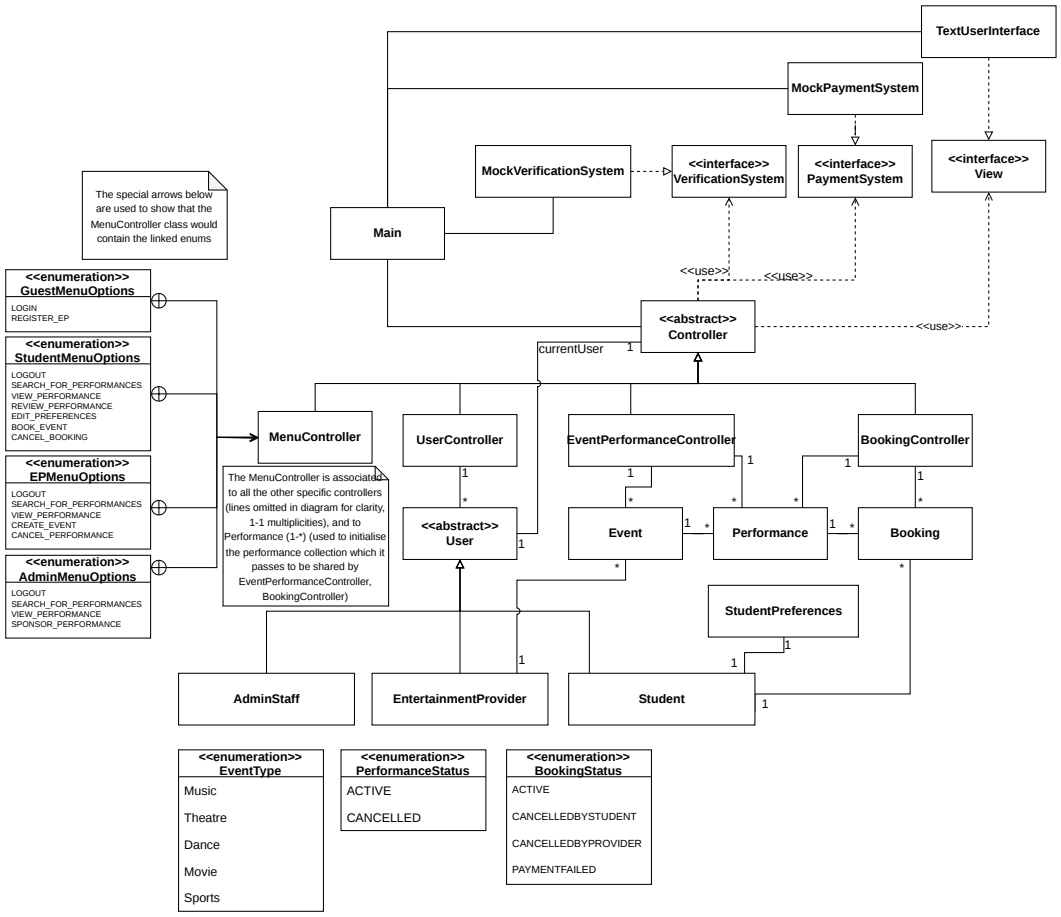

Figure 1: UML Class Diagram – high level view

for students by the sponsored amount, but otherwise payment works normally. The actual payment of the sponsorship amount happens externally from our system.

- Is EP business number a String or an int?
  Assumption: A string, as it could contain characters.

2 Task 2: UML class model

A class model is shown in Fig. 1 (showing only class associations) and Fig. 2, Fig. 3, Fig. 4, and Fig. 5(showing the fields and operations that would go with the classes). Please note that the fields do not contain those which are objects of other classes, which are represented through associations.

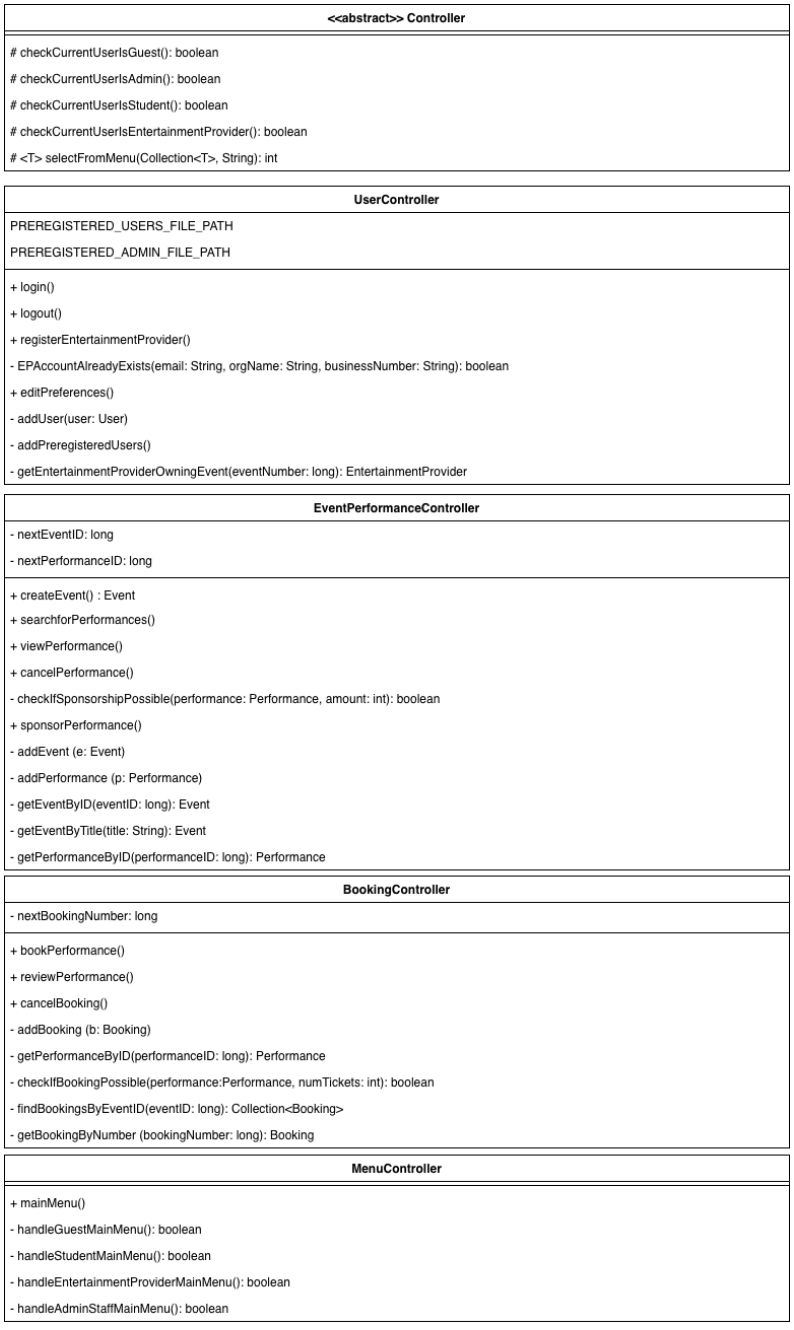

Figure 2: UML Class Diagram – classes in detail 1

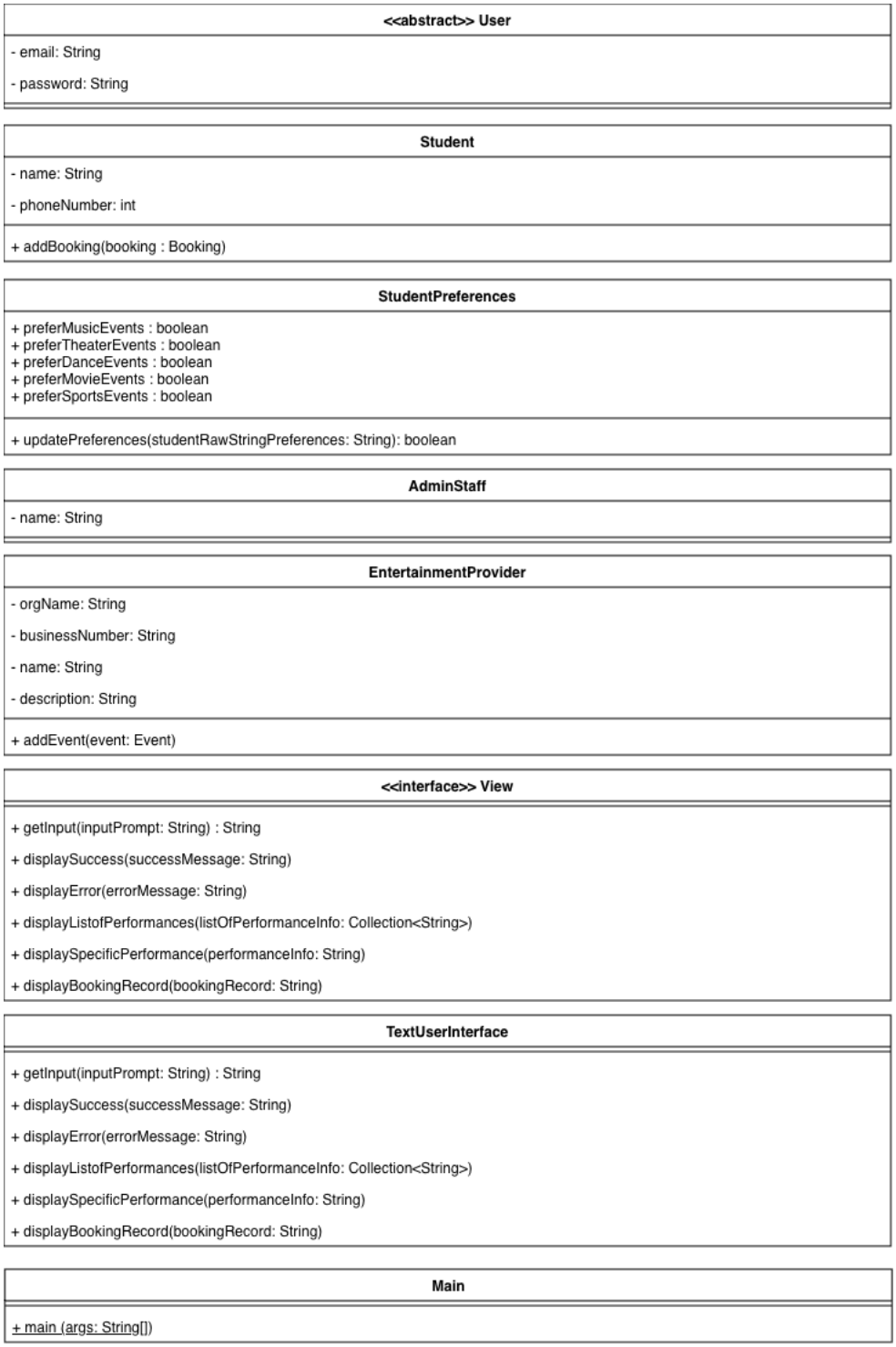

Figure 3: UML Class Diagram – classes in detail 2

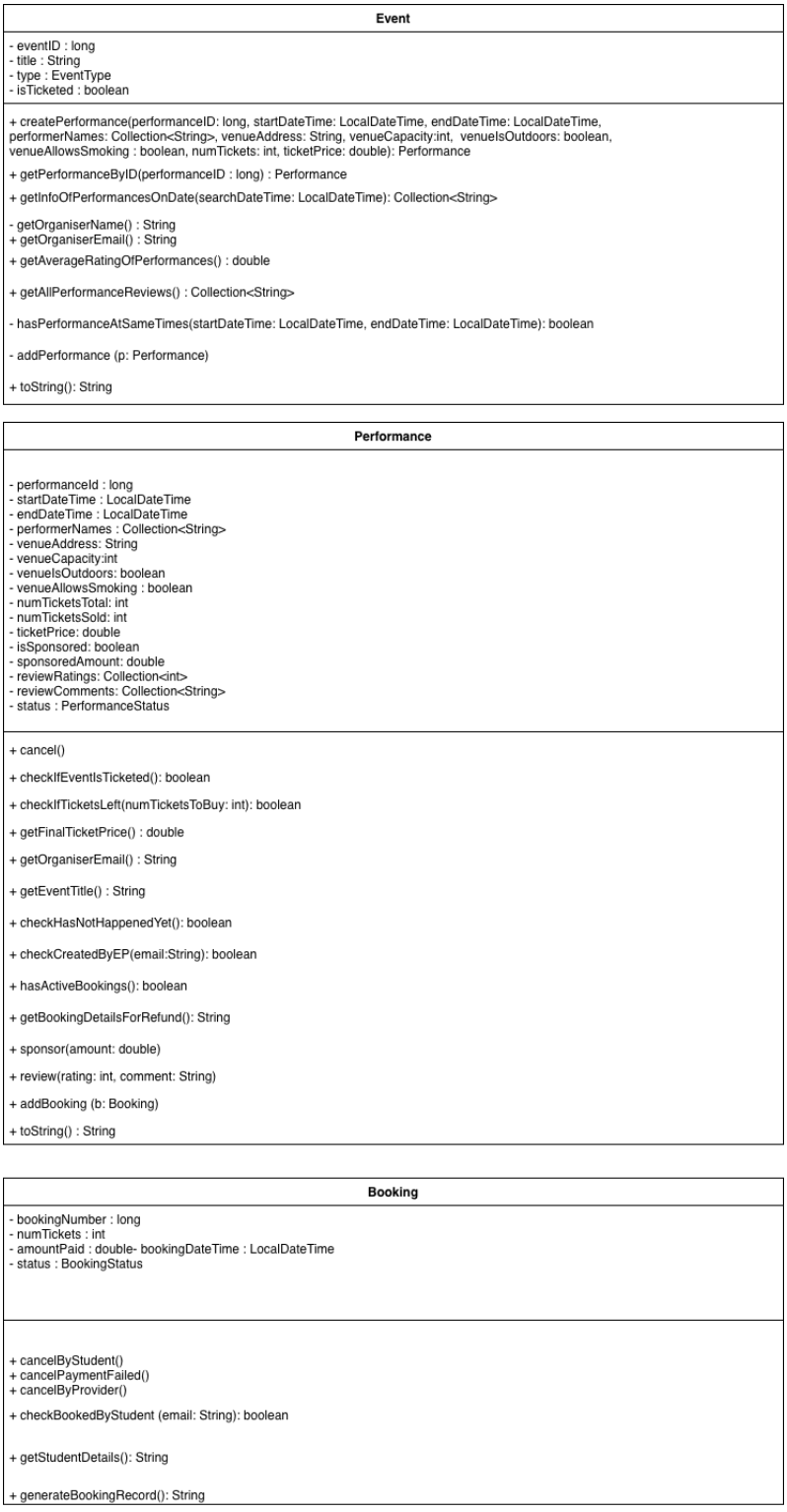

Figure 4: UML Class Diagram – classes in detail 3

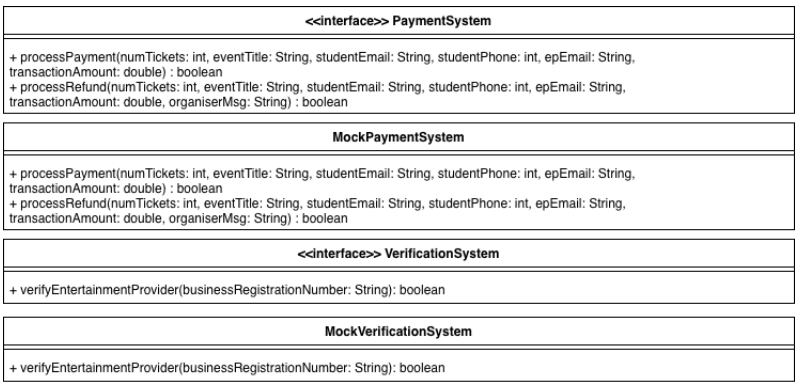

Figure 5: UML Class Diagram – classes in detail 4

3 Task 3: High-level description of the UML class model

Notes about the class model:

- We've omitted constructors from the diagram (as well as getters and setters), but they should implicitly be there.
- According to the requirements, whenever an error happens due to incorrect input the system should prompt the user for new input and continue the use case until the user provides correct input—that's what was expected as a solution.
- We need to be able to keep track of the current user, e.g. because some actions are only doable by certain user types - we do this in Controller (since it's connected to everything).
- At the very start of the program, the Main would create a MenuController object first, with a null current user. The menu controller would then initialise the currentUser (empty), and create one object of each specific controller with it. It would pass to the EventPerformanceController and BookingController an empty collection of performances.
- The user chooses an action to run as follows: MenuController checks the type of the current user, and uses the "selectFromMenu" method to show a list of possible actions appropriate for that type (defined by the enumerations inside MenuController) to the user. The method "selectFromMenu" uses the TextUserInterface's "getInput" method to input the user's choice. Then, the MenuController calls the appropriate method in a specific controller that matches the user's chosen action.
- When doing the "Search for Performances" and "View Performance" use cases, we essentially loop through all Performances on the chosen date, and convert each Perfromance to a string before passing it back to EventPerformanceController and then to View. You could pass Performances directly, but by converting them to a

string first we avoid having to associate Performance to View, reducing coupling.

Explanations regarding design decisions:

- Alternatively to our diagram, specific controllers could be used for each different user (e.g. StudentController, EPController, etc).
- Splitting the Event entity into 'Ticketed event' and 'Non-ticketed event' is possible, but the resulting entities would be quite empty, so it would not be ideal.
- The amount paid for a booking needs to be stored such that consumers who bought tickets before sponsorship is introduced can be refunded correctly.
- No information is needed for the external systems as our system communicates with them only through messages.
- Each class has distinct responsibilities and well-defined interfaces given by their public operations (decomposition, modularisation design principles).
- An attempt is made at keeping high cohesion by delegating responsibilities to the right classes (e.g. controllers to handle collections of objects, objects which are entities to handle information about themselves).
- Even if sometimes being able for objects to communicate would have simplified the implementation, associations were kept to those that are conceptually correct and those that are absolutely needed for the functioning of the system. Moreover, when only certain information (e.g attribute values) about an object several steps in a chain of associations was needed, that information, and not the entire object at the other end, is passed between the objects, to avoid creating additional associations (Law of Demeter, also see sequence diagrams). This is all meant to keep coupling low.
- Attributes are always private and only the needed (public) getter and setter operations for them (not shown in class diagram) would be used. Moreover, only those operations that must be accessible by other objects are public, while operations for internal processing are private. These help respect the design principles of abstraction and encapsulation.
- The MVC pattern was used. Given the requirements, they can be interpreted as there not being multiple views so MVC is arguably unnecessary at this stage of the development (one could still keep the controllers without the View class), so not using it would also be acceptable. However, it's easy to imagine the need for multiple views for the system in the future—for example, if we wanted to show the system as both a website and a phone app. To be robust to future changes, we introduce MVC now.
- Singletons could have been used for keeping event, performance, and bookings ID's, which are correctly kept unique system-wide. However, these would have posed difficulties when testing (objects would have had numbers associated to them after

each test), and moreover can be easily handled by the controllers storing as an attribute the next number to use. In addition, singletons could be used for logging.

- Observer is arguably not applicable anywhere.
- Command could be used in theory, but there's no need for it, and it would make the diagram unnecessarily complicated.

4 Task 4: UML sequence diagram

The three required sequence diagrams from the coursework instructions are provided in Fig. 6, Fig. 7, and Fig. 8.

Note that we have considered implementation details and further strengthened the sequence diagrams when possible, even beyond requirements. This is to guide your CW3, but this does not have an impact on how your CW2 will be marked. For instance, in the Book Performance use case, we now consider that the system will continue prompting the student until they give a performance ID that is valid, for a ticketed event, and for a performance that still has tickets left. The latter two checks were not specified in the previous iteration of requirements. Similarly, it makes sense to have a booking record for the same use case, to keep track of bookings in the system. That is why we include that operation as well.

5 Task 5: Low-level design

Note that: AcmeCorp are still considering your solution to Task 5. You do not need to include it nor build upon it for CW3.

1) We have included a copy of the code to implement the solution here.

2) The list of assumptions below is not exhaustive.

- We assume that business numbers can only contain alphanumeric characters in the Latin alphabet. This assumption might be incorrect in certain locations.
- We assume that the government and business registration name (except for the null case) have been sanitised (to protect our system from security issues) at the UI component.
- We assume that the functions that call BusinessVerifier.verify(...) accept a boolean as a return value.

3) A service-based solution will help us quickly scale. A stateless solution in 1) ensures that we can have such distribution architecture.

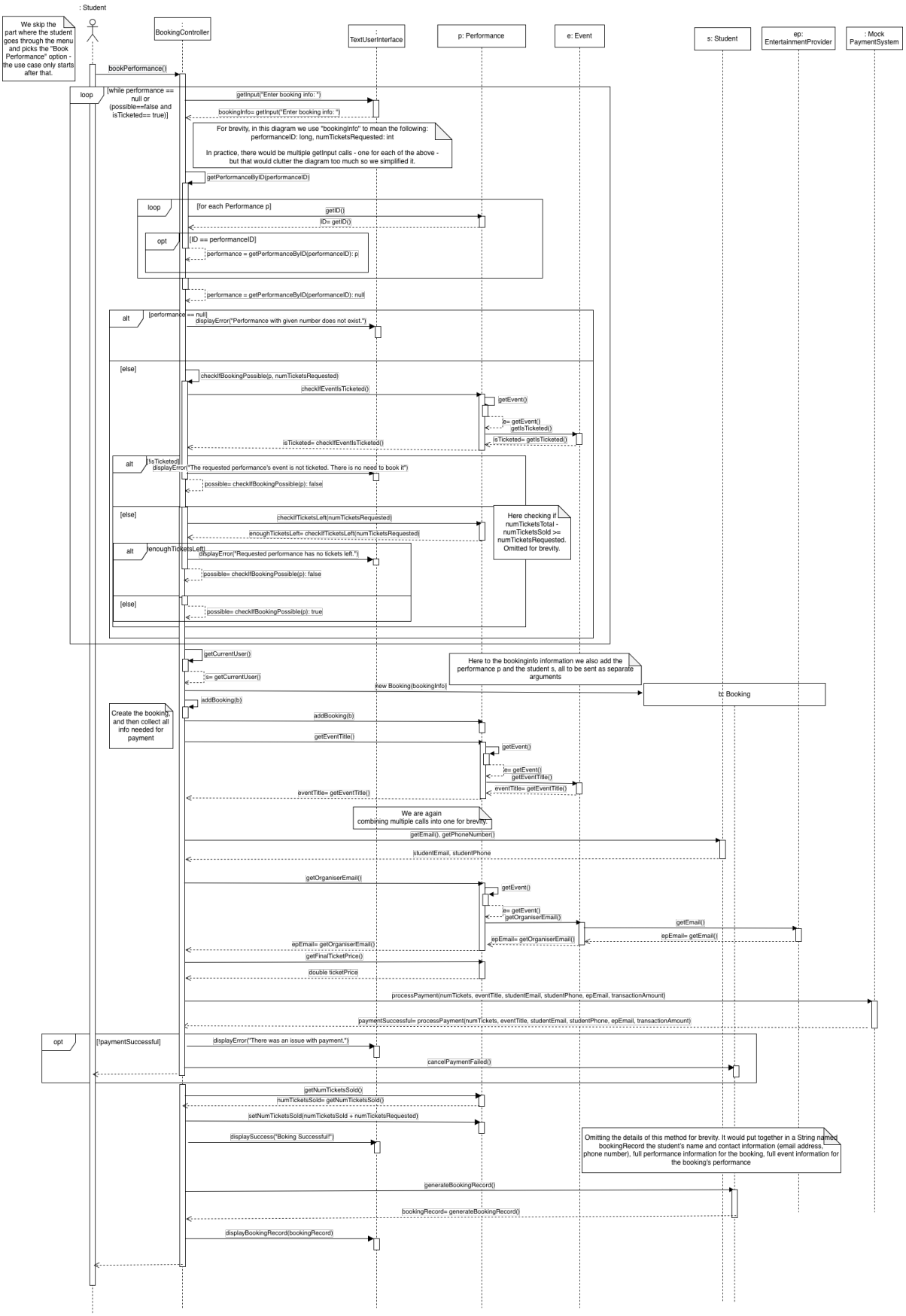

Figure 6: UML Sequence Diagram for "Book Performance"

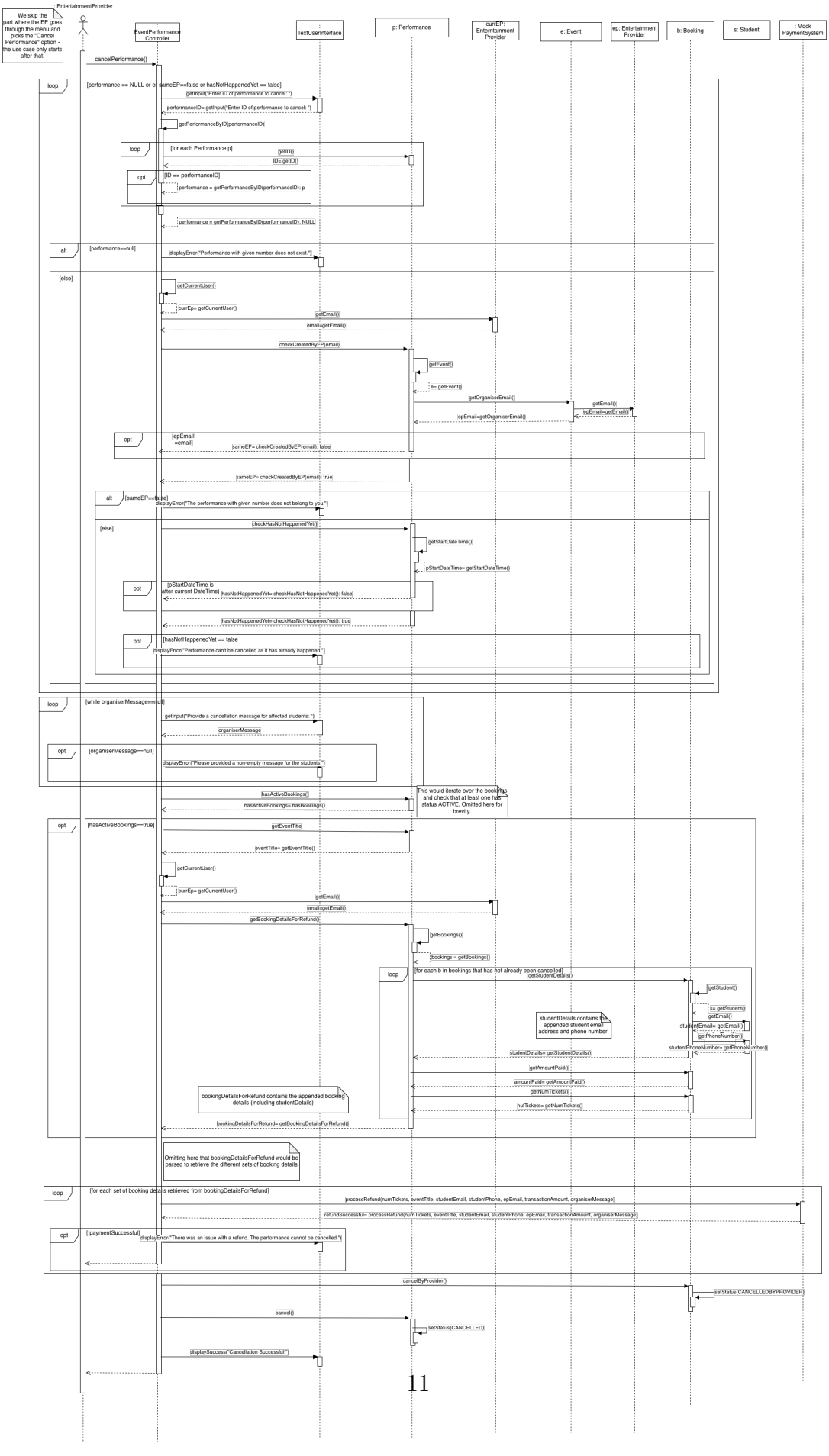

Figure 7: UML Sequence Diagram for “Cancel Performance”

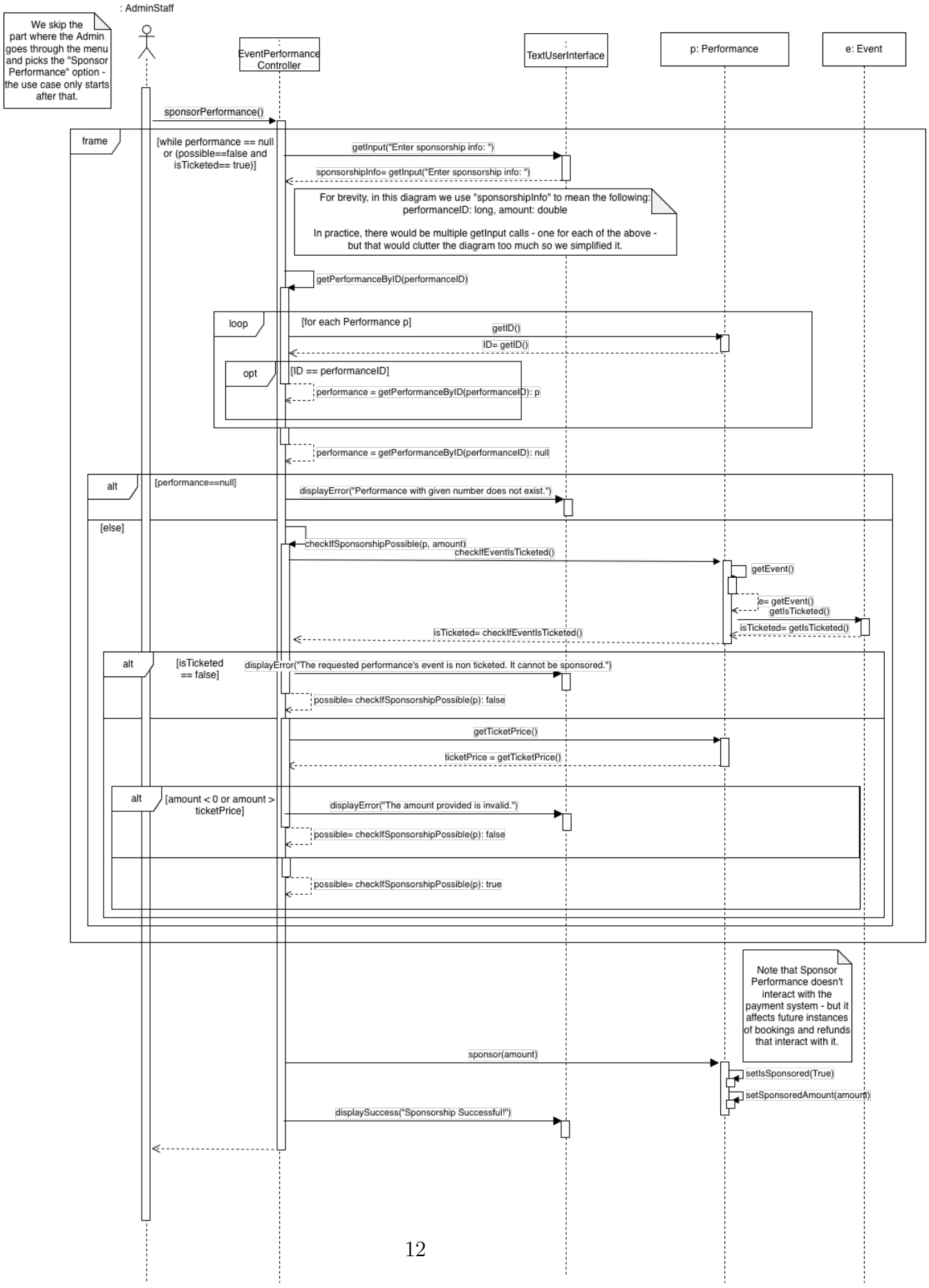

Figure 8: UML Sequence Diagram for “Sponsor Performance”